

Paralelismo ao Nível de Instrução (ILP)

Dependências de dados e níveis de otimização em C

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

2026

1 Enunciado

Implementar três laços em C para investigar ILP: inicializar um vetor com um cálculo simples, somar seus elementos com dependência entre iterações e repetir a soma com múltiplos acumuladores. Comparar versões compiladas com -O0, -O2 e -O3, analisando o efeito do estilo do código e das dependências no desempenho.

2 Fundamentação teórica

2.1 O que é ILP

Uma CPU superescalar consegue iniciar mais de uma instrução por ciclo, desde que elas não dependam umas das outras. Essa sobreposição se chama paralelismo em nível de instrução (ILP). O limite prático vem das dependências de dados: se a instrução B precisa do resultado da instrução A, o processador espera A terminar antes de começar B.

2.2 Latência, throughput e por que acumuladores importam

Uma soma inteira (ADD) em processadores x86 modernos tem latência de 1 ciclo e throughput de 1 operação por ciclo por porta. Quando o laço usa um acumulador único, cada soma depende do resultado da soma anterior: $s_{i+1} = s_i + v[i + 1]$. Isso cria uma cadeia de N somas sequenciais, onde cada uma espera 1 ciclo pela anterior. O processador tem capacidade de emitir várias somas por ciclo, mas a dependência impede.

Com quatro acumuladores independentes (s_0, s_1, s_2, s_3), cada cadeia tem $N/4$ somas. As quatro cadeias não dependem entre si, então o processador pode emitir somas de cadeias diferentes no mesmo ciclo. O caminho crítico cai de N para $N/4$ ciclos, um speedup teórico de até $4\times$.

2.3 O laço de inicialização

O laço $v[i] = 3*i + 1$ escreve posições diferentes do vetor a cada iteração. Não há dependência entre escritas consecutivas, então o compilador pode vetorizar o laço com instruções SIMD e o processador pode executar várias escrituras em paralelo.

3 Experimento

3.1 Configuração

O vetor contém $N = 100.000.000$ inteiros (int, 4 bytes cada), totalizando aproximadamente 381,5 MiB (400 MB). Esse valor foi dimensionado com base na arquitetura de teste (processador Intel Core i7-14700HX): (1) supera em mais de $11\times$ o cache L3 compartilhado de 33 MiB (e os caches L1D de 48 KiB e L2 de 2 MiB por núcleo), garantindo que o experimento atravesse toda a hierarquia de memória até a DRAM; (2) mantém os tempos de execução na faixa de 26 a 300 ms, minimizando o peso relativo do ruído de medição e do escalonador do sistema operacional; e (3) consome apenas uma fração da memória RAM principal, sem risco de paginação em disco (*swap*).

A função $v[i] = 3*i + 1$ preenche o vetor. Depois, dois laços somam todos os elementos: o primeiro com um acumulador, o segundo com quatro.

Cada laço é cronometrado isoladamente com `clock_gettime(CLOCK_MONOTONIC)`, que tem resolução de nanossegundos e não conta tempo de outros processos. O programa compara os dois resultados para garantir que a soma com múltiplos acumuladores produz o mesmo valor.

3.2 Compilação

O código foi compilado três vezes, variando apenas o nível de otimização:

```
1 gcc -std=c11 -Wall -Wextra -O0 tarefa03.c -o tarefa03_00
2 gcc -std=c11 -Wall -Wextra -O2 tarefa03.c -o tarefa03_02
3 gcc -std=c11 -Wall -Wextra -O3 tarefa03.c -o tarefa03_03
```

-O0 desliga todas as otimizações do compilador, o que preserva a estrutura original do código. -O2 aplica desenrolamento de laços e alocação de registradores. -O3 adiciona vetorização automática com instruções SIMD.

4 Código-fonte

```
1 #define _POSIX_C_SOURCE 199309L
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h>
6
7 #define DEFAULT_N 100000000L
8
9 static double agora(void) {
10     struct timespec t;
11     clock_gettime(CLOCK_MONOTONIC, &t);
12     return t.tv_sec + t.tv_nsec / 1e9;
13 }
14
15 int main(int argc, char **argv) {
16     long n = argc > 1 ? atol(argv[1]) : DEFAULT_N;
```

```

17  if (n <= 0) {
18      fprintf(stderr, "Uso: %s [numero_de_elementos]\n", argv[0]);
19      return 1;
20  }
21
22  int *v = malloc((size_t)n * sizeof(*v));
23  if (!v) {
24      perror("malloc");
25      return 1;
26  }
27
28  double inicio = agora();
29  for (long i = 0; i < n; ++i)
30      v[i] = (int)(3 * i + 1);
31  double tempo_inicializacao = agora() - inicio;
32
33  inicio = agora();
34  long long soma_dependente = 0;
35  for (long i = 0; i < n; ++i)
36      soma_dependente += v[i];
37  double tempo_dependente = agora() - inicio;
38
39  inicio = agora();
40  long long s0 = 0, s1 = 0, s2 = 0, s3 = 0;
41  long i = 0;
42  for (; i + 3 < n; i += 4) {
43      s0 += v[i];
44      s1 += v[i + 1];
45      s2 += v[i + 2];
46      s3 += v[i + 3];
47  }
48  for (; i < n; ++i)
49      s0 += v[i];
50  long long soma_multiplos = s0 + s1 + s2 + s3;
51  double tempo_multiplos = agora() - inicio;
52
53  printf("N=%ld inicializacao=%.9f dependente=%.9f multiplos=%.9f "
54        "somas=%lld,%lld\n", n, tempo_inicializacao,
55        tempo_dependente, tempo_multiplos, soma_dependente,
56        soma_multiplos);
57  free(v);
58  return soma_dependente != soma_multiplos;
59 }

```

Listing 1: tarefa03.c

5 Resultados

5.1 Tempos medidos

Table 1: Tempo de execução em segundos ($N = 100.000.000$).

Otimização	Inicialização	Soma dependente	Múltiplos acumuladores
O0	0,300583	0,151139	0,058475
O2	0,102487	0,028251	0,026156
O3	0,109349	0,026172	0,030444

5.2 Speedup dos múltiplos acumuladores

Table 2: Speedup = tempo_{dependente} / tempo_{múltiplos}.

Otimização	Speedup
O0	2,58×
O2	1,08×
O3	0,86×

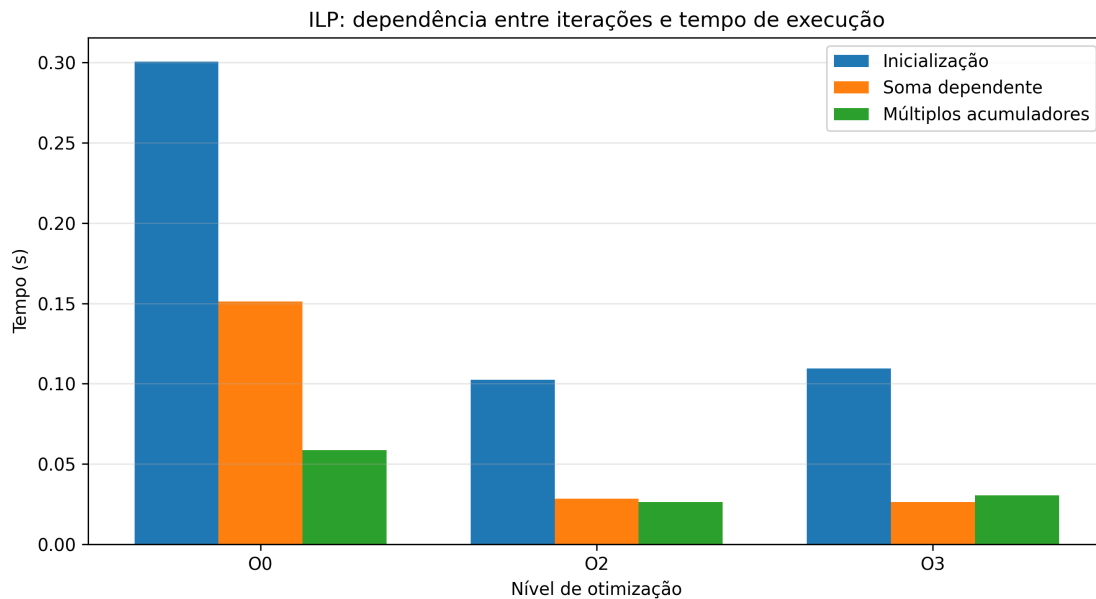


Figure 1: Tempo de execução dos três laços por nível de otimização ($N = 100.000.000$).

5.3 Leitura dos resultados

Com -O0, a soma com quatro acumuladores rodou em 0,058 s contra 0,151 s do acumulador único: ganho de 2,58× de desempenho. O limite teórico de aceleração seria de 4× (quatro cadeias independentes de cálculo). Esse teto não é integralmente atingido porque a taxa de leitura da memória (load de cada elemento do vetor) também impõe um piso ao tempo total de processamento.

Com -02, o compilador aplica desenrolamento de laço e alocação agressiva de registradores na versão com acumulador único. O ganho da intervenção manual cai para $1,08\times$, pois o compilador reestrutura o fluxo de instruções automaticamente.

Com -03, os tempos de soma dependente (0,026 s) e de múltiplos acumuladores (0,030 s) tornam-se equivalentes, com variação residual na ordem de 4 ms. Nesse nível de otimização, a vetorização automática SIMD e a reordenação profunda atuam em ambas as abordagens, fazendo com que o gargalo de execução seja dominado pela taxa de transferência de dados da memória e não pela dependência aritmética original.

A inicialização reduziu de 0,301 s em -00 para cerca de 0,102 s em -02. Como o laço de inicialização grava elementos contíguos em posições independentes de memória, o compilador consegue aplicar vetorização SIMD com amplo aproveitamento de largura de banda.

6 Discussão

6.1 Caminho crítico e dependências de dados

Os laços de soma executam o mesmo número de operações aritméticas, $O(N)$, porém com caminhos críticos distintos. No acumulador único, cada iteração depende do resultado imediato da anterior ($s_{i+1} = s_i + v[i + 1]$), criando uma cadeia estritamente sequencial de N somas que impede a sobreposição de instruções.

Com quatro acumuladores, as operações são distribuídas em quatro cadeias mutuamente independentes de comprimento $N/4$. Isso permite que a arquitetura superescalar despache múltiplas instruções de adição por ciclo de clock. A soma final dos acumuladores parciais ($s_0 + s_1 + s_2 + s_3$) requer apenas três adições adicionais, custo irrelevante para $N = 10^8$.

6.2 Atuação das otimizações do compilador

Nos níveis -02 e -03, o compilador aplica técnicas avançadas de reordenação, desenrolamento de laços e vetorização vetorial SIMD. Esse processo elimina a desvantagem do código sequencial direto, equiparando o desempenho da versão ingênua ao da versão manualmente desenrolada.

6.3 Escopo do paralelismo investigado

O paralelismo analisado ocorre exclusivamente em nível de instrução (ILP) dentro de um único núcleo de execução da CPU, por meio de unidades funcionais paralelas e pipelines superescalares. Esse mecanismo opera internamente no hardware e não deve ser confundido com paralelismo entre threads ou múltiplos núcleos físicos.

7 Conclusão

O experimento comprovou que dependências de dados entre iterações consecutivas restringem diretamente a capacidade de execução paralela do pipeline. Sem otimização do compilador (-00),

a divisão em quatro acumuladores independentes proporcionou um ganho de $2,58\times$ na soma do vetor.

Com níveis de otimização elevados (-O2 e -O3), o compilador foi capaz de identificar e explorar automaticamente esse paralelismo, tornando equivalente o desempenho de ambas as implementações. Assim, comprova-se que a exposição de instruções independentes é essencial para o desempenho quando o código depende da estrutura direta dada pelo programador.